

Curso de Programación en Python

Para Estudiantes de Secundaria

Prof. César Rodríguez

13 de agosto de 2026

Resumen

Este curso introduce a los estudiantes de secundaria en el mundo de la programación utilizando el lenguaje Python. A lo largo de 12 módulos se desarrollan los fundamentos de la programación: variables, tipos de datos, estructuras de control, colecciones, funciones y un proyecto final integrador. Cada módulo combina teoría breve, ejemplos resueltos y ejercicios propuestos con distintos niveles de dificultad, de modo que el estudiante aprende haciendo. No se requiere experiencia previa en programación ni conocimientos avanzados de matemáticas.

Índice

1. Introducción a la Programación	2
1.1. ¿Qué es programar?	2
1.2. ¿Qué es Python?	2
1.3. Instalación y primer programa	2
1.3.1. Instalación en Windows	2
1.3.2. Instalación en Linux	3
1.3.3. Alternativa en línea	3
1.4. El primer programa: “Hola mundo”	3
1.4.1. ¿Cómo se ejecuta?	3
1.4.2. Comentarios en el código	3
1.5. Ejercicios propuestos	4
2. Variables y Tipos de Datos	5
2.1. ¿Qué es una variable?	5
2.2. Reglas para nombrar variables	5
2.3. Tipos de datos básicos	5
2.4. Cambio de tipo de datos (casting)	6
2.5. Ejercicios propuestos	7
3. Entrada de Datos por Teclado	7
3.1. La función input()	7
3.2. Programa con entrada numérica	8
3.3. Ejercicios propuestos	8

4. Operadores y Expresiones	9
4.1. Operadores aritméticos	9
4.2. Operadores de comparación	10
4.3. Operadores lógicos	10
4.4. Precedencia de operadores	11
4.5. Ejercicios propuestos	12
5. Estructuras Condicionales	13
5.1. La sentencia if	13
5.2. La sentencia elif	13
5.3. Condiciones anidadas	14
5.4. Ejercicios propuestos	14
6. Bucles: for y while	15
6.1. ¿Por qué necesitamos bucles?	15
6.2. El bucle for	15
6.3. El bucle while	16
6.4. Acumuladores	16
6.5. Ejercicios propuestos	17
7. Cadenas de Texto	18
7.1. ¿Qué es una cadena de texto?	18
7.2. Rebanadas (slicing)	18
7.3. Métodos útiles de cadenas	19
7.4. Ejercicios propuestos	19
8. Listas	20
8.1. ¿Qué es una lista?	20
8.2. Métodos y funciones útiles para listas	21
8.3. Recorrer una lista	21
8.4. Ejercicios propuestos	22
9. Tuplas y Diccionarios	23
9.1. Tuplas	23
9.2. Diccionarios	24
9.3. Métodos útiles de diccionarios	25
9.4. Ejercicios propuestos	25
10. Funciones	27
10.1. ¿Qué es una función?	27
10.2. Definir y llamar funciones	27
10.3. Parámetros y argumentos	27
10.4. Ámbito de las variables	28
10.5. Ejercicios propuestos	29
11. Biblioteca Estándar: math y random	30
11.1. ¿Qué es la biblioteca estándar?	30
11.2. El módulo math	30
11.3. El módulo random	30

11.4. Ejercicios propuestos	31
12. Proyecto Final	32
12.1. Propuesta de proyectos	33
12.2. Criterios de evaluación	34
13. Referencias	34

1. Introducción a la Programación

1.1. ¿Qué es programar?

Programar consiste en escribir **instrucciones** para que una computadora realice una tarea. Un **algoritmo** es la secuencia ordenada de pasos que se necesitan para resolver un problema.

Ejemplo

Imagina que quieres darle instrucciones a un amigo para preparar un sándwich. Los pasos podrían ser:

1. Conseguir pan, queso y jamón.
2. Cortar el pan por la mitad.
3. Colocar el queso y el jamón entre las dos mitades.
4. Servir el sándwich.

Ese conjunto de pasos ordenados es un **algoritmo**. Una computadora necesita que le demos instrucciones mucho más detalladas, en un lenguaje que ella pueda entender: un **lenguaje de programación**.

1.2. ¿Qué es Python?

Python es un lenguaje de programación creado por Guido van Rossum a finales de los años 80. Algunas de sus características más importantes son:

- **Fácil de leer:** su sintaxis es clara y cercana al lenguaje humano.
- **Fácil de escribir:** se necesitan pocas líneas para hacer mucho.
- **Muy usado:** se emplea en ciencia de datos, inteligencia artificial, robótica, desarrollo web y videojuegos, entre otros campos.
- **Gratis y multiplataforma:** funciona en Windows, Linux y macOS.

Recuerda

La versión estable de Python es la **3.14** (año 2026). Todas las computadoras de esta era usan Python 3; Python 2 ya no se usa.

1.3. Instalación y primer programa

1.3.1. Instalación en Windows

1. Ingresa a <https://www.python.org/downloads/>.
2. Descarga el instalador de la última versión (Python 3.14).
3. Ejecuta el instalador y **marca la casilla** “Add Python to PATH”.
4. Haz clic en “Install Now”.

1.3.2. Instalación en Linux

Python suele venir preinstalado. Para verificarlo, abre una terminal y escribe:

```
1 python3 --version
```

Si no está instalado, usa el gestor de paquetes de tu distribución:

```
1 sudo apt install python3
```

1.3.3. Alternativa en línea

Si no puedes instalar Python en tu computadora, puedes usar **Replit** (replit.com), **Google Colab** (colab.research.google.com) o **Python Tutor** (pythontutor.com) para practicar sin instalar nada.

1.4. El primer programa: “Hola mundo”

El programa más sencillo que se puede escribir en Python es:

```
1 print("Hola, mundo!")
```

Listing 1: Nuestro primer programa

Cuando se ejecuta, el programa muestra en pantalla:

```
1 Hola, mundo!
```

1.4.1. ¿Cómo se ejecuta?

1. Abre el **IDLE** (viene instalado con Python) o un editor como Thonny o VS Code.
2. Escribe el programa anterior en un archivo nuevo.
3. Guarda el archivo con extensión `.py` (por ejemplo, `hola.py`).
4. Ejecuta el programa con la tecla **F5** (en IDLE) o con el botón “Run”.

Concepto clave

`print()` es una **función** que muestra texto en la pantalla. El texto va entre comillas simples `'` o dobles `"`. Casi todos los programas que escribamos comenzarán usando `print()`.

1.4.2. Comentarios en el código

Un **comentario** es una nota que escribimos para los humanos que lean el código. La computadora lo ignora. En Python los comentarios empiezan con el símbolo `#`:

```
1 # Este es mi primer programa
2 # Muestra un saludo en pantalla
3 print("Hola, mundo!")
4 print("Bienvenidos al curso de Python")
```

Listing 2: Programa con comentarios

1.5. Ejercicios propuestos

Ejercicio

Ejercicio 1 [Fácil]

Escribe un programa que muestre tu nombre completo en pantalla, por ejemplo:

```
Mi nombre es Ana López
```

Ejercicio

Ejercicio 2 [Fácil]

Escribe un programa que muestre tres líneas distintas:

1. Tu nombre.
2. Tu ciudad.
3. Tu hobby favorito.

Cada dato debe aparecer en una línea separada usando tres llamadas a `print()`.

Ejercicio

Ejercicio 3 [Fácil]

Escribe un programa que muestre un rectángulo dibujado con asteriscos, como este:

```
*****
*****
*****
```

Ayuda: cada línea del rectángulo es una llamada a `print()`.

Ejercicio

Ejercicio 4 [Intermedio]

Escribe un programa con al menos tres comentarios que explique, paso a paso, qué hace el programa “Hola mundo”. No es necesario que el programa haga algo nuevo; la idea es practicar la escritura de comentarios claros.

Reto

Reto 1 [Difícil]

Investiga qué hace la función `print()` cuando recibe dos argumentos separados por coma, por ejemplo:

```
print("Hola", "mundo")
```

Escribe un programa que use esta idea para imprimir tu nombre y tu edad en la misma línea, separados por un espacio. Explica con un comentario qué observaste.

2. Variables y Tipos de Datos

2.1. ¿Qué es una variable?

Una **variable** es una **caja** en la memoria de la computadora donde guardamos un valor. A cada caja le ponemos una etiqueta (un **nombre**) para poder encontrarla después.

Ejemplo

```
1 edad = 15
2 nombre = "Ana"
3 estatura = 1.62
4
5 print(edad)
6 print(nombre)
7 print(estatura)
```

Listing 3: Creando variables

Aquí guardamos:

- el número entero 15 en la variable `edad`;
- el texto "Ana" en la variable `nombre`;
- el número decimal 1.62 en la variable `estatura`.

2.2. Reglas para nombrar variables

- Los nombres no pueden empezar con un número: `2edad` es inválido.
- No pueden contener espacios: usa guión bajo `_`: `mi_edad`.
- No pueden usar palabras reservadas de Python (como `print`, `if`, `for`).
- Distinguen mayúsculas y minúsculas: `Edad` y `edad` son variables diferentes.
- Es buena práctica usar nombres descriptivos: `promedio_notas` es mejor que `p`.

Recuerda

El símbolo `=` en programación **no** significa “igual que”. Significa “**asignar**” o “guardar en”. La variable de la izquierda recibe el valor de la derecha.

2.3. Tipos de datos básicos

En Python, cada valor tiene un **tipo**. Los más usados son:

Para saber el tipo de una variable usamos la función `type()`:

```
1 edad = 15
2 print(type(edad))           # <class 'int'>
3
4 pi = 3.14
5 print(type(pi))             # <class 'float'>
```

Tipo	Nombre	Ejemplo	Descripción
Entero	int	42	Números sin parte decimal
Decimal	float	3.14	Números con parte decimal
Texto	str	"Hola"	Cadenas de caracteres
Lógico	bool	True	Solo dos valores: Verdadero o Falso

```

6
7 nombre = "Ana"
8 print(type(nombre))           # <class 'str'>
9
10 es_mayor = True
11 print(type(es_mayor))        # <class 'bool'>

```

Listing 4: La función type()

2.4. Cambio de tipo de datos (casting)

Podemos convertir un valor de un tipo a otro:

```

1 numero_texto = "10"
2 print(type(numero_texto))      # <class 'str'>
3
4 numero = int(numero_texto)     # "10" se convierte en 10
5 print(type(numero))           # <class 'int'>
6 print(numero + 5)              # 15
7
8 # De decimal a entero (se truncan los decimales)
9 precio = 9.99
10 precio_entero = int(precio)
11 print(precio_entero)          # 9
12
13 # De número a texto
14 anio = 2026
15 texto = str(anio)
16 print(texto + " es el año actual") # 2026 es el año actual

```

Listing 5: Conversión de tipos

Recuerda

Las funciones de conversión más usadas son:

- `int(valor)`: convierte a entero.
- `float(valor)`: convierte a decimal.
- `str(valor)`: convierte a texto.
- `bool(valor)`: convierte a lógico.

2.5. Ejercicios propuestos

Ejercicio

Ejercicio 5 [Fácil]

Crea tres variables: una con tu edad, una con tu nombre y una con tu estatura. Luego imprime cada una con `print()`, y además imprime el tipo de cada variable usando `type()`.

Ejercicio

Ejercicio 6 [Fácil]

Escribe un programa que calcule y muestre el área de un rectángulo de base 8 y altura 5. Usa dos variables (`base` y `altura`) y otra variable `area` para guardar el resultado. Recuerda que el área se calcula como $\text{base} \times \text{altura}$.

Ejercicio

Ejercicio 7 [Intermedio]

Encuentra el error en el siguiente programa y corrígelo:

```
1 nombre = "Carlos"
2 edad = 14
3 print("Hola, me llamo" + nombre + " y tengo" + edad + " años")
```

Ayuda: ¿qué tipo de dato espera la suma con `+`? Ejecuta el programa, observa el mensaje de error y explica la solución en un comentario.

Ejercicio

Ejercicio 8 [Intermedio]

Escribe un programa que intercambie el valor de dos variables. Por ejemplo, si `a = 5` y `b = 3`, al final debe quedar `a = 3` y `b = 5`. Imprime los valores antes y después. Ayuda: usa una variable auxiliar (una “tercera caja”).

Reto

Reto 2 [Difícil]

Escribe un programa que calcule y muestre el perímetro de una circunferencia. Usa una variable `radio = 7` y el valor de $\pi = 3.14159$. La fórmula del perímetro es $P = 2 \cdot \pi \cdot \text{radio}$. Muestra el resultado con un mensaje descriptivo, por ejemplo: El perímetro es 43.98226.

3. Entrada de Datos por Teclado

3.1. La función `input()`

Hasta ahora los programas siempre mostraban los mismos valores. Para que el programa **interactúe** con el usuario, usamos la función `input()`, que lee lo que el usuario escribe por teclado.

Ejemplo

```
1 nombre = input("¿Cómo te llamas? ")
2 print("Mucho gusto,", nombre)
```

Listing 6: Saludando al usuario

Cuando se ejecuta, el programa espera a que el usuario escriba algo y presione **Enter**. Luego responde:

```
1 ¿Cómo te llamas? Ana
2 Mucho gusto, Ana
```

Recuerda

Importante: `input()` siempre devuelve un **texto** (`str`), aunque el usuario escriba un número. Si necesitamos operar con números, debemos convertir el valor con `int()` o `float()`.

3.2. Programa con entrada numérica

Ejemplo

```
1 numero1 = int(input("Escribe el primer número: "))
2 numero2 = int(input("Escribe el segundo número: "))
3
4 suma = numero1 + numero2
5 print("La suma es:", suma)
```

Listing 7: Suma de dos números

Nota que `input()` se **envuelve** en `int(...)` para convertir el texto en número. Si no lo hiciéramos, la operación `numero1 + numero2` sería una concatenación de textos (por ejemplo, `"5" + "3"` daría `"53"`).

3.3. Ejercicios propuestos

Ejercicio

Ejercicio 9 [Fácil]

Escribe un programa que pregunte al usuario su nombre y su edad, y luego muestre:

```
1 Hola, [nombre]. El próximo año tendrás [edad + 1] años.
```

Ayuda: la edad se lee como texto; conviértela con `int()` para poder sumarle 1.

Ejercicio

Ejercicio 10 [Fácil]

Escribe un programa que pida dos números enteros y muestre su suma, su resta, su multiplicación y su división. Por cada operación imprime una línea descriptiva, por ejemplo:

```
1 5 + 3 = 8
2 5 - 3 = 2
3 5 * 3 = 15
4 5 / 3 = 1.6666666666666667
```

Ejercicio

Ejercicio 11 [Intermedio]

Escribe un programa que pregunte la base y la altura de un triángulo y calcule su área usando la fórmula $A = \frac{b \cdot h}{2}$. Muestra el resultado con un mensaje claro. Ayuda: usa `float()` para aceptar números con decimales.

Ejercicio

Ejercicio 12 [Intermedio]

Escribe un programa que pregunte el precio de un producto y calcule el precio con el 12% de IVA incluido. El programa debe mostrar el precio original, el monto del IVA y el precio final.

Reto

Reto 3 [Difícil]

Escribe un programa que pregunte cuántos segundos tiene una duración y la convierta en **horas, minutos y segundos**. Por ejemplo, 3661 segundos son 1 hora, 1 minuto y 1 segundo. Ayuda: usa la división entera `//` y el módulo `%` (el residuo de la división). Prueba tu programa con 10000 segundos.

4. Operadores y Expresiones

4.1. Operadores aritméticos

Operador	Nombre	Ejemplo	Resultado
+	Suma	7 + 3	10
-	Resta	7 - 3	4
*	Multiplicación	7 * 3	21
/	División	7 / 3	2.333...
//	División entera	7 // 3	2
%	Módulo (residuo)	7 % 3	1
**	Potencia	2 ** 5	32

Concepto clave

El **módulo** (%) devuelve el residuo de una división entera. Por ejemplo, $10 \% 3$ es 1 porque 10 entre 3 es 3 y sobran 1. Es muy útil para saber si un número es par: si $\text{numero} \% 2$ es 0, el número es par.

4.2. Operadores de comparación

Estos operadores comparan dos valores y devuelven un resultado lógico: **True** (verdadero) o **False** (falso).

Operador	Significado	Ejemplo
<code>==</code>	Igual que	$5 == 5 \rightarrow \text{True}$
<code>!=</code>	Diferente de	$5 != 3 \rightarrow \text{True}$
<code>></code>	Mayor que	$5 > 8 \rightarrow \text{False}$
<code><</code>	Menor que	$5 < 8 \rightarrow \text{True}$
<code>>=</code>	Mayor o igual que	$5 >= 5 \rightarrow \text{True}$
<code><=</code>	Menor o igual que	$5 <= 4 \rightarrow \text{False}$

Recuerda

No confundas `=` (asignar un valor a una variable) con `==` (comparar si dos valores son iguales).

4.3. Operadores lógicos

Permiten combinar condiciones:

Operador	Significado	Resultado
<code>and</code>	Y	Verdadero solo si ambas condiciones son verdaderas
<code>or</code>	O	Verdadero si al menos una condición es verdadera
<code>not</code>	NO	Invierte el valor: verdadero se convierte en falso y viceversa

Ejemplo

```
1 edad = 16
2 tiene_permiso = True
3
4 # and: las dos deben ser verdaderas
5 puede_entrar = (edad >= 18) and tiene_permiso
6 print(puede_entrar)           # False, porque 16 no es >= 18
7
8 # or: basta con que una sea verdadera
9 puede_ver_pelicula = (edad >= 13) or tiene_permiso
10 print(puede_ver_pelicula)    # True
11
12 # not: invierte el valor
13 no_es_mayor = not (edad >= 18)
14 print(no_es_mayor)          # True
```

Listing 8: Usando operadores lógicos

4.4. Precedencia de operadores

Cuando una expresión tiene varios operadores, Python los evalúa en un orden establecido (como en matemáticas):

1. Paréntesis ()
2. Potencias **
3. Multiplicación, división y módulo * / //%
4. Suma y resta + -
5. Comparaciones == != < > <= >=
6. Lógicos and, or, not

Recuerda

Cuando tengas dudas, **usa paréntesis**. El código `(2 + 3) * 4` es más claro que `2 + 3 * 4` y evita errores de interpretación.

4.5. Ejercicios propuestos

Ejercicio

Ejercicio 13 [Fácil]

Sin ejecutar el programa, escribe en tu cuaderno qué mostrará cada `print()`. Luego verifica con Python.

```
1 print(15 // 4)
2 print(15 % 4)
3 print(2 ** 10)
4 print(10 - 2 * 3)
5 print((10 - 2) * 3)
6 print(9 % 2)
```

Ejercicio

Ejercicio 14 [Fácil]

Escribe un programa que le pida al usuario un número y muestre si el número es par o impar. Ayuda: un número es par si `numero % 2` es igual a 0.

Ejercicio

Ejercicio 15 [Intermedio]

Escribe un programa que pida tres notas de un estudiante (de 0 a 20) y calcule el promedio. Luego muestra el promedio con dos decimales. Ayuda: usa la función `round(promedio, 2)` para redondear.

Ejercicio

Ejercicio 16 [Intermedio]

Escribe un programa que pida un número de 4 cifras y muestre sus dígitos por separado. Por ejemplo, si el usuario escribe 4521, el programa debe mostrar:

```
Dígitos: 4 5 2 1
```

Ayuda: divide sucesivamente con `//` y `%`. Por ejemplo, `4521 // 1000 = 4` y `4521 % 1000 = 521`.

Reto

Reto 4 [Difícil]

Escribe un programa que pida un número entero de tres cifras e indique si es un **número capicúa** (se lee igual de izquierda a derecha que de derecha a izquierda). Por ejemplo, 121 es capicúa, pero 123 no lo es. Ayuda: descompón el número en centenas, decenas y unidades usando `//` y `%`.

5. Estructuras Condicionales

5.1. La sentencia if

Un programa no siempre hace lo mismo. A veces necesita **tomar decisiones**. La estructura `if` permite ejecutar un bloque de código **solo si** una condición es verdadera.

Ejemplo

```
1 edad = int(input("¿Cuántos años tienes? "))
2
3 if edad >= 18:
4     print("Eres mayor de edad")
5 else:
6     print("Aún eres menor de edad")
```

Listing 9: Primer condicional

Si el usuario escribe 20, el programa muestra “Eres mayor de edad”. Si escribe 14, muestra “Aún eres menor de edad”.

Recuerda

- Después de `if` (y de `else`), la línea termina con **dos puntos** `:`.
- El bloque que se ejecuta debe estar **sangrado** (indentado) con espacios o tabulaciones. Python usa la sangría para saber qué instrucciones pertenecen a cada bloque.
- Por convención se usan **4 espacios** de sangría.

5.2. La sentencia elif

Cuando hay más de dos opciones, usamos `elif` (contracción de “else if”).

Ejemplo

```
1 nota = int(input("Escribe tu nota (0 a 20): "))
2
3 if nota >= 18:
4     print("Sobresaliente")
5 elif nota >= 15:
6     print("Notable")
7 elif nota >= 11:
8     print("Aprobado")
9 else:
10    print("Reprobado")
```

Listing 10: Varias opciones con elif

Python evalúa las condiciones **en orden**: si la primera es verdadera, ejecuta su bloque y se salta las demás.

5.3. Condiciones anidadas

Dentro de un bloque `if` podemos escribir otro `if`. Esto se llama **anidamiento**.

Ejemplo

```
1 edad = int(input("¿Cuántos años tienes? "))
2
3 if edad >= 12:
4     if edad >= 18:
5         print("Eres un adulto")
6     else:
7         print("Eres un adolescente")
8 else:
9     print("Eres un niño o niña")
```

Listing 11: Condicionales anidados

5.4. Ejercicios propuestos

Ejercicio

Ejercicio 17 [Fácil]

Escribe un programa que pida un número y diga si es **positivo**, **negativo** o **cero**.

Ejercicio

Ejercicio 18 [Fácil]

Escribe un programa que pida un número entero y muestre si es **par** o **impar**, y además si es **mayor que 100** o **menor o igual que 100**. Usa condiciones anidadas.

Ejercicio

Ejercicio 19 [Intermedio]

Escribe un programa que pida dos números y muestre cuál es el **mayor**. Si son iguales, debe mostrar el mensaje “Los números son iguales”.

Ejercicio

Ejercicio 20 [Intermedio]

Escribe un programa que pida tres números y los muestre **en orden ascendente** (del menor al mayor). Ayuda: compara los tres números usando `if` y `elif`, y ordena las condiciones de menor a mayor.

Reto

Reto 5 [Difícil]

Crea un pequeño **asistente de clasificación de edades**:

- De 0 a 2 años: “Bebé”.
- De 3 a 11 años: “Niño(a)”.
- De 12 a 17 años: “Adolescente”.
- De 18 a 59 años: “Adulto”.
- 60 o más: “Adulto mayor”.
- Si es menor que 0, muestra “Edad inválida”.

Usa **and** para combinar los límites inferiores y superiores de cada rango.

6. Bucles: for y while

6.1. ¿Por qué necesitamos bucles?

Un **bucle** (o **laço de repetición**) permite ejecutar un bloque de código **muchas veces** sin escribirlo muchas veces. Por ejemplo, para mostrar los números del 1 al 10 no es necesario escribir diez `print()`.

6.2. El bucle for

El bucle **for** recorre una secuencia de elementos (por ejemplo, un rango de números) y ejecuta el bloque una vez por cada elemento.

Ejemplo

```
1 for numero in range(1, 6):  
2     print(numero)
```

Listing 12: Mostrar números del 1 al 5

Salida:

```
1 1  
2 2  
3 3  
4 4  
5 5
```

Nota que `range(1, 6)` genera los números del 1 al 5: el **inicio es inclusivo** y el **final es exclusivo** (no se incluye el 6).

Recuerda

- `range(n)` genera números de 0 a `n-1`.
- `range(inicio, fin)` genera de `inicio` a `fin-1`.
- `range(inicio, fin, paso)` genera saltando de `paso` en `paso`.

6.3. El bucle while

El bucle **while** repite un bloque **mientras** una condición sea verdadera. Es útil cuando no sabemos de antemano cuántas veces se repetirá.

Ejemplo

```
1 contador = 1
2
3 while contador <= 5:
4     print(contador)
5     contador = contador + 1
```

Listing 13: Contar hasta 5 con while

El programa muestra los números del 1 al 5. La línea `contador = contador + 1` es **esencial**: sin ella, la condición nunca cambiaría y el bucle se repetiría **infinitamente**.

Recuerda

- **Cuidado con los bucles infinitos**: si la condición de **while** nunca se vuelve falsa, el programa nunca termina.
- Una forma abreviada de `contador = contador + 1` es `contador += 1`.
- La sentencia **break** sale del bucle inmediatamente.
- La sentencia **continue** salta a la siguiente iteración.

6.4. Acumuladores

Un patrón muy común es usar una variable para **acumular** la suma de varios valores.

Ejemplo

```
1 suma = 0
2
3 for numero in range(1, 101):
4     suma = suma + numero
5
6 print("La suma del 1 al 100 es:", suma)    # 5050
```

Listing 14: Sumar los primeros 100 números

La variable `suma` inicia en 0 y en cada vuelta recibe su valor anterior más el número actual.

6.5. Ejercicios propuestos

Ejercicio

Ejercicio 21 [Fácil]

Escribe un programa con `for` que muestre los números **pares** del 2 al 20. Ayuda: usa `range(2, 21, 2)`.

Ejercicio

Ejercicio 22 [Fácil]

Escribe un programa que muestre la **tabla de multiplicar** de un número que pide al usuario. Por ejemplo, si el usuario escribe 7, debe mostrar:

```
1 7 x 1 = 7
2 7 x 2 = 14
3 ...
4 7 x 10 = 70
```

Ejercicio

Ejercicio 23 [Intermedio]

Escribe un programa que pida al usuario **5 calificaciones** (una por una) y calcule su promedio. Usa un bucle `for` y un acumulador. Al final muestra el promedio con dos decimales.

Ejercicio

Ejercicio 24 [Intermedio]

Escribe un programa que cuente de 10 hacia atrás hasta 1 (con `for` y `range`) y que, al terminar, muestre el mensaje “¡Despegue!”.

Ejercicio

Ejercicio 25 [Intermedio]

Escribe un programa con `while` que pida al usuario números enteros y los sume, pero que **se detenga** cuando el usuario escriba 0. Al final, muestra la suma de todos los números escritos.

Reto

Reto 6 [Difícil]

Escribe un programa que calcule el **factorial** de un número. El factorial de n (escrito $n!$) es el producto de todos los enteros desde 1 hasta n . Por ejemplo:

- $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$
- $7! = 5040$

Pide el número al usuario y muestra el resultado. Ayuda: usa un acumulador que inicia en 1 y un bucle que vaya de 1 hasta el número.

7. Cadenas de Texto

7.1. ¿Qué es una cadena de texto?

Una **cadena** (`str`) es una secuencia de caracteres. Podemos acceder a cada carácter mediante su **índice**. Los índices empiezan en **0**.

Ejemplo

```
1 palabra = "Python"
2
3 print(palabra[0])      # P
4 print(palabra[1])      # y
5 print(palabra[5])      # n
6 print(len(palabra))    # 6 (cantidad de caracteres)
```

Listing 15: Acceder a caracteres

Los índices negativos cuentan desde el final:

```
1 print(palabra[-1])     # n (el último)
2 print(palabra[-2])     # o (el penúltimo)
```

7.2. Rebanadas (slicing)

Con los dos puntos `:` podemos extraer una **porción** de la cadena.

Ejemplo

```
1 texto = "Hola mundo"
2
3 print(texto[0:4])      # Hola    (del 0 al 3)
4 print(texto[5:10])     # mundo   (del 5 al 9)
5 print(texto[:4])       # Hola    (del inicio al 3)
6 print(texto[5:])       # mundo   (del 5 al final)
7 print(texto[:])        # Hola mundo (toda la cadena)
```

Listing 16: Rebanadas

`cadena[inicio:fin]` toma desde `inicio` hasta `fin-1`.

Método	Qué hace
<code>cadena.upper()</code>	Convierte todo a mayúsculas
<code>cadena.lower()</code>	Convierte todo a minúsculas
<code>cadena.capitalize()</code>	Mayúscula inicial, resto en minúsculas
<code>cadena.strip()</code>	Elimina espacios al inicio y al final
<code>cadena.count("x")</code>	Cuenta cuántas veces aparece x
<code>cadena.find("x")</code>	Devuelve el índice de la primera aparición de x
<code>cadena.replace("a","b")</code>	Reemplaza todas las a por b
<code>cadena.split(" ")</code>	Divide la cadena en una lista según el separador
<code>"-".join(lista)</code>	Une los elementos de una lista con el separador

7.3. Métodos útiles de cadenas

Ejemplo

```

1 mensaje = "  hola mundo  "
2
3 print(mensaje.upper())           # "  HOLA MUNDO  "
4 print(mensaje.strip())          # "hola mundo"
5 print(mensaje.strip().upper())  # "HOLA MUNDO"
6
7 texto = "ana,pedro,luís"
8 nombres = texto.split(",")
9 print(nombres)                  # ['ana', 'pedro', 'luís']

```

Listing 17: Usando métodos

Los métodos se pueden **encadenar**: primero `strip()` y luego `upper()`.

7.4. Ejercicios propuestos

Ejercicio

Ejercicio 26 [Fácil]

Escribe un programa que pida una palabra y muestre:

- La palabra en mayúsculas.
- La palabra en minúsculas.
- El número de letras que tiene.
- Su primera y última letra.

Ejercicio

Ejercicio 27 [Fácil]

Escribe un programa que pida el nombre completo de una persona y lo muestre con **mayúscula inicial en cada palabra**. Ayuda: investiga el método `title()`.

Ejercicio

Ejercicio 28 [Intermedio]

Escribe un programa que pida una frase y cuente cuántas veces aparece la letra **a** (sin importar si es mayúscula o minúscula). Ayuda: convierte la frase a minúsculas con `lower()` antes de contar.

Ejercicio

Ejercicio 29 [Intermedio]

Escribe un programa que pida una palabra e indique si es un **palíndromo** (se lee igual al derecho y al revés). Ejemplos: **reconocer**, **radar**, **ana**. Ayuda: compara la palabra con su inversa usando `palabra[::-1]`.

Ejercicio

Ejercicio 30 [Intermedio]

Escribe un programa que pida una frase y la muestre con las palabras **en orden inverso**. Por ejemplo, la frase “me gusta programar” debe mostrar “programar gusta me”. Ayuda: usa `split()` y `join()`.

Reto

Reto 7 [Difícil]

Escribe un programa que **cifre** un mensaje con el **cifrado César**: cada letra se desplaza 3 posiciones en el alfabeto. Por ejemplo, la **a** se convierte en **d**, la **b** en **e**, y así sucesivamente (la **z** vuelve a **a**). Pide al usuario una frase y muestra la versión cifrada. Ayuda: investiga las funciones `ord()` y `chr()`, que convierten caracteres en números y viceversa.

8. Listas

8.1. ¿Qué es una lista?

Una **lista** es una colección **ordenada** y **modificable** de valores. Se crea con corchetes `[ ]` y sus elementos se separan con comas.

Ejemplo

```
1 frutas = ["manzana", "pera", "uva", "mango"]
2
3 print(frutas)           # ['manzana', 'pera', 'uva', 'mango']
4 print(frutas[0])        # manzana
5 print(frutas[2])        # uva
6 print(len(frutas))      # 4
7
8 # Las listas son modificables
9 frutas[1] = "banana"
10 print(frutas)           # ['manzana', 'banana', 'uva', 'mango']
11
12 # Agregar un elemento al final
13 frutas.append("kiwi")
14 print(frutas)           # ['manzana', 'banana', 'uva', 'mango',
15                          # 'kiwi']
16
17 # Quitar el último elemento
18 ultimo = frutas.pop()
19 print(ultimo)           # kiwi
20 print(frutas)
```

Listing 18: Creando y usando listas

8.2. Métodos y funciones útiles para listas

Operación	Qué hace
<code>lista.append(x)</code>	Agrega x al final
<code>lista.insert(i, x)</code>	Inserta x en la posición i
<code>lista.remove(x)</code>	Elimina la primera aparición de x
<code>lista.pop()</code>	Elimina y devuelve el último elemento
<code>lista.sort()</code>	Ordena la lista de menor a mayor (en su lugar)
<code>sorted(lista)</code>	Devuelve una copia ordenada sin modificar la original
<code>sum(lista)</code>	Suma todos los elementos (si son números)
<code>min(lista)</code>	Devuelve el elemento menor
<code>max(lista)</code>	Devuelve el elemento mayor
<code>lista.count(x)</code>	Cuenta cuántas veces aparece x
<code>x in lista</code>	Devuelve True si x está en la lista

8.3. Recorrer una lista

Podemos recorrer todos los elementos de una lista con `for`:

Ejemplo

```
1 nombres = ["Ana", "Luis", "Marta", "Pedro"]
2
3 for nombre in nombres:
4     print("Hola,", nombre)
5
6 # También podemos acceder por índice con range
7 for i in range(len(nombres)):
8     print(i, "->", nombres[i])
```

Listing 19: Recorriendo una lista

8.4. Ejercicios propuestos

Ejercicio

Ejercicio 31 [Fácil]

Crea una lista con 5 colores. Luego:

1. Muestra el primer y el último color.
2. Agrega un sexto color con `append()`.
3. Reemplaza el segundo color por otro.
4. Muestra la lista final y su tamaño con `len()`.

Ejercicio

Ejercicio 32 [Fácil]

Escribe un programa que pida al usuario **5 números** (uno por uno), los guarde en una lista y luego muestre la suma y el promedio. Ayuda: usa `append()` dentro de un bucle `for`.

Ejercicio

Ejercicio 33 [Intermedio]

Escribe un programa que pida al usuario **3 notas** (de 0 a 20), las guarde en una lista y muestre:

- El promedio.
- La nota más alta (`max`).
- La nota más baja (`min`).
- Las notas ordenadas de mayor a menor.

Ejercicio

Ejercicio 34 [Intermedio]

Escribe un programa que, a partir de una lista de números, cuente cuántos son **pares** y cuántos son **impares**. Por ejemplo, con la lista [4, 7, 2, 9, 10, 5], debe mostrar "3 pares y 3 impares".

Ejercicio

Ejercicio 35 [Intermedio]

Escribe un programa que pida nombres y los guarde en una lista hasta que el usuario escriba "fin". Luego muestra:

- Cuántos nombres se ingresaron.
- El nombre más largo.
- La lista ordenada alfabéticamente.

Reto

Reto 8 [Difícil]

Escribe un programa que genere la **secuencia de Fibonacci** hasta un límite indicado por el usuario. En la secuencia de Fibonacci cada número es la suma de los dos anteriores: 0, 1, 1, 2, 3, 5, 8, 13, 21, ... Guarda los primeros términos en una lista y muéstrala. Ayuda: comienza con [0, 1] y ve agregando la suma de los dos últimos.

9. Tuplas y Diccionarios

9.1. Tuplas

Una **tupla** es como una lista, pero **no se puede modificar** (es **inmutable**). Se crea con paréntesis ().

Ejemplo

```
1 punto = (3, 5)
2 print(punto[0])           # 3
3 print(punto[1])           # 5
4
5 # Esto da ERROR: las tuplas no se pueden modificar
6 # punto[0] = 10
7
8 # Una tupla con un solo elemento necesita una coma
9 unico = (7,)
10 print(unico)
```

Listing 20: Creando una tupla

Desempaquetado: podemos asignar cada elemento de una tupla a una variable:

```
1 coordenadas = (4, 7)
2 x, y = coordenadas
3 print("x =", x)          # x = 4
4 print("y =", y)          # y = 7
```

9.2. Diccionarios

Un **diccionario** guarda pares de **clave: valor**. Se crea con llaves { } y permite buscar un valor a partir de su clave.

Ejemplo

```
1 estudiante = {
2     "nombre": "Ana López",
3     "edad": 15,
4     "curso": "3ro Secundaria"
5 }
6
7 print(estudiante["nombre"])    # Ana López
8 print(estudiante["edad"])      # 15
9
10 # Modificar un valor
11 estudiante["edad"] = 16
12 print(estudiante["edad"])      # 16
13
14 # Agregar una nueva clave
15 estudiante["ciudad"] = "Sucre"
16 print(estudiante)
```

Listing 21: Creando un diccionario

Operación	Qué hace
<code>dic["clave"]</code>	Devuelve el valor de la clave (error si no existe)
<code>dic.get("clave")</code>	Devuelve el valor o <code>None</code> si no existe
<code>dic.get("clave", x)</code>	Devuelve <code>x</code> si la clave no existe
<code>dic.keys()</code>	Devuelve todas las claves
<code>dic.values()</code>	Devuelve todos los valores
<code>dic.items()</code>	Devuelve pares (clave, valor)
<code>"clave" in dic</code>	Devuelve <code>True</code> si la clave existe
<code>dic["clave"] = v</code>	Agrega o modifica el valor de la clave

9.3. Métodos útiles de diccionarios

Ejemplo

```

1 capitales = {
2     "Bolivia": "La Paz",
3     "Argentina": "Buenos Aires",
4     "Colombia": "Bogotá",
5     "Perú": "Lima"
6 }
7
8 for pais, capital in capitales.items():
9     print("La capital de", pais, "es", capital)

```

Listing 22: Recorriendo un diccionario

9.4. Ejercicios propuestos

Ejercicio

Ejercicio 36 [Fácil]

Crea un diccionario con los datos de un libro: título, autor y año de publicación. Luego:

1. Muestra el título.
2. Modifica el año.
3. Agrega la clave **género**.
4. Muestra todas las claves y todos los valores.

Ejercicio

Ejercicio 37 [Fácil]

Crea una tupla con las 4 estaciones del año. Muestra la primera y la última estación, y luego la cantidad de elementos con `len()`. Escribe un comentario explicando por qué no podemos modificar el primer elemento.

Ejercicio

Ejercicio 38 [Intermedio]

Escribe un programa que pida al usuario 3 países y su capital (un país a la vez), los guarde en un diccionario y luego muestre las capitales de todos los países ingresados.

Ejercicio

Ejercicio 39 [Intermedio]

Dado el diccionario de calificaciones {"Ana": 18, "Luis": 14, "Marta": 20, "Pedro": 11}, escribe un programa que:

- Muestre el promedio de las calificaciones.
- Muestre quién tiene la calificación más alta.
- Muestre quién tiene la más baja.

Ayuda: usa `dic.values()`, `max()` y `min()`. Para saber quién tiene la nota, busca el estudiante cuya calificación coincida.

Ejercicio

Ejercicio 40 [Intermedio]

Escribe un programa que **cuenta las letras** de una palabra usando un diccionario. Por ejemplo, para la palabra "banana", el resultado debe ser:

```
{'b': 1, 'a': 3, 'n': 2}
```

Ayuda: recorre cada letra; si la letra ya está en el diccionario, suma 1 a su valor; si no, la agrega con valor 1.

Reto

Reto 9 [Difícil]

Escribe un programa que gestione una **agenda telefónica** con un diccionario. El programa debe:

1. Permitir agregar contactos (nombre y número).
2. Permitir buscar el número de un contacto.
3. Permitir eliminar un contacto.
4. Mostrar todos los contactos.

El menú se muestra en un bucle `while` y se sale con la opción "salir". Usa `input()` para elegir la opción.

10. Funciones

10.1. ¿Qué es una función?

Una **función** es un bloque de código **reutilizable** que realiza una tarea concreta. Nos permite escribir el código una sola vez y usarlo muchas veces. Ya hemos usado funciones como `print()`, `input()`, `len()` y `int()`; ahora aprenderemos a crear las nuestras.

10.2. Definir y llamar funciones

Para crear una función usamos la palabra clave `def`:

Ejemplo

```
1 def saludar():
2     print("¡Hola! Bienvenido al curso")
3
4 # Llamar a la función
5 saludar()
6 saludar()
7 saludar()
```

Listing 23: Mi primera función

El programa muestra el mensaje tres veces. La definición **no ejecuta** el código; solo se ejecuta cuando **llamamos** a la función.

10.3. Parámetros y argumentos

Una función puede recibir **parámetros** (datos de entrada) para trabajar con ellos.

Ejemplo

```
1 def saludar_persona(nombre):
2     print("Hola,", nombre)
3
4 def sumar(a, b):
5     resultado = a + b
6     return resultado
7
8 saludar_persona("Ana")
9 saludar_persona("Luis")
10
11 # La función sumar devuelve un valor con return
12 total = sumar(5, 3)
13 print("La suma es:", total)
```

Listing 24: Función con parámetros

- nombre, a y b son **parámetros**: variables locales de la función.
- **return** devuelve un resultado para que el programa que llamó a la función pueda usarlo.

Recuerda

Valores por defecto: un parámetro puede tener un valor inicial que se usa si la función se llama sin ese argumento:

```
1 def saludar(nombre, saludo="Hola"):
2     print(saludo + ", " + nombre)
3
4 saludar("Ana") # Hola, Ana
5 saludar("Luis", "Buenos días") # Buenos días, Luis
```

10.4. Ámbito de las variables

Las variables creadas dentro de una función son **locales**: solo existen dentro de la función. Las variables creadas fuera son **globales** y son visibles en todo el programa.

Ejemplo

```
1 mensaje = "Hola desde fuera"      # variable global
2
3 def cambiar_mensaje():
4     mensaje_local = "Hola desde dentro" # variable local
5     print(mensaje_local)
6     print(mensaje)      # la global también es visible
7
8 cambiar_mensaje()
9 print(mensaje)          # funciona, es global
10 # print(mensaje_local) # ERROR: no existe fuera de la función
```

Listing 25: Variables locales y globales

10.5. Ejercicios propuestos

Ejercicio

Ejercicio 41 [Fácil]

Escribe una función llamada `dibujar_linea()` que muestre una línea de 20 asteriscos. Luego llámala 3 veces para dibujar tres líneas.

Ejercicio

Ejercicio 42 [Fácil]

Escribe una función `cuadrado(n)` que reciba un número y devuelva su cuadrado. Pide un número al usuario y muestra el resultado.

Ejercicio

Ejercicio 43 [Intermedio]

Escribe una función `es_par(n)` que devuelva `True` si el número es par y `False` si es impar. Luego escribe un programa que pida 5 números y cuente cuántos son pares usando la función.

Ejercicio

Ejercicio 44 [Intermedio]

Escribe una función `promedio(notas)` que reciba una lista de notas y devuelva su promedio. Luego pide al usuario 4 notas, guárdalas en una lista y muestra el promedio usando la función.

Ejercicio

Ejercicio 45 [Intermedio]

Escribe una función `invertir_texto(texto)` que devuelva el texto invertido (por ejemplo, “hola” se convierte en “aloh”). Luego escribe un programa que pida una frase, la invierta con la función y muestre el resultado.

Reto

Reto 10 [Difícil]

Crea una **calculadora con funciones**. Define las funciones `sumar(a, b)`, `restar(a, b)`, `multiplicar(a, b)` y `dividir(a, b)`. Luego escribe un programa que muestre un menú con las 4 operaciones, pida dos números y la operación elegida, y muestre el resultado. Usa un bucle `while` para que el menú se repita hasta que el usuario elija “salir”.

11. Biblioteca Estándar: math y random

11.1. ¿Qué es la biblioteca estándar?

Python incluye cientos de **módulos** con funciones listas para usar. Un módulo es un archivo con funciones y herramientas relacionadas. Para usarlo, lo importamos con la instrucción `import`.

11.2. El módulo math

Proporciona funciones matemáticas avanzadas:

Ejemplo

```
1 import math
2
3 print(math.sqrt(144))           # 12.0   (raíz cuadrada)
4 print(math.pi)                 # 3.141592653589793
5 print(math.ceil(4.2))           # 5     (redondea hacia arriba)
6 print(math.floor(4.8))          # 4     (redondea hacia abajo)
7 print(math.pow(2, 10))          # 1024.0 (potencia)
8 print(math.factorial(5))        # 120
9 print(math.fabs(-7))            # 7.0   (valor absoluto)
```

Listing 26: Usando math

11.3. El módulo random

Permite generar **números aleatorios**, muy útil para juegos:

Ejemplo

```
1 import random
2
3 print(random.randint(1, 6))      # número entero entre 1 y 6 (dado
4                                  )
5 print(random.random())           # número decimal entre 0 y 1
6 print(random.choice(["rojo", "verde", "azul"])) # elige un
7                                   elemento al azar
8 print(random.shuffle())          # mezcla una lista en su lugar
9
10 lista = [1, 2, 3, 4, 5]
11 random.shuffle(lista)
12 print(lista)                    # lista mezclada
```

Listing 27: Usando random

Recuerda

`random.randint(a, b)` incluye ambos extremos: `randint(1, 6)` puede devolver 1 o 6.

11.4. Ejercicios propuestos

Ejercicio

Ejercicio 46 [Fácil]

Escribe un programa que use `math` para calcular y mostrar:

- La raíz cuadrada de 289.
- El valor de π redondeado a 4 decimales.
- El factorial de 10.

Ejercicio

Ejercicio 47 [Fácil]

Escribe un programa que simule el lanzamiento de un dado: genera un número aleatorio entre 1 y 6 y lo muestra. Pide al usuario presionar **Enter** para lanzar, y que el programa lance el dado 3 veces mostrando cada resultado.

Ejercicio

Ejercicio 48 [Intermedio]

Escribe el juego “Adivina el número”:

1. El programa elige un número aleatorio entre 1 y 50.
2. El usuario intenta adivinarlo.
3. Si el número es menor o mayor, el programa da una pista.
4. Cuando acierta, muestra en cuántos intentos lo logró.

Usa un bucle `while` que se repita hasta acertar.

Ejercicio

Ejercicio 49 [Intermedio]

Escribe un programa que use `math` para calcular la **hipotenusa** de un triángulo rectángulo. Pide los dos catetos y usa la fórmula $c = \sqrt{a^2 + b^2}$. Ayuda: usa `math.sqrt()` y `math.pow()` (o `** 2`).

Ejercicio

Ejercicio 50 [Intermedio]

Escribe un programa que simule `random.choice` para elegir al azar la respuesta a una pregunta. Define una lista de respuestas como `["Sí", "No", "Quizás", "Sin duda"]` y muestra una cada vez que el usuario haga una pregunta.

Reto

Reto 11 [Difícil]

Escribe el juego “Piedra, papel o tijera” contra la computadora:

1. El usuario elige piedra, papel o tijera.
2. La computadora elige una al azar con `random.choice`.
3. El programa determina quién gana (piedra vence a tijera, tijera vence a papel, papel vence a piedra).
4. Se juegan 5 rondas y al final se muestra el marcador.

Ayuda: usa `if` y `elif` para todas las combinaciones posibles.

12. Proyecto Final

El proyecto final integra todo lo aprendido. Cada estudiante debe entregar **un** programa completo, bien documentado, que funcione sin errores y que combine al menos tres temas del curso.

12.1. Propuesta de proyectos

Ejercicio

Proyecto A: Agenda de contactos [Intermedio]

Crea un programa que gestione una agenda de contactos con un diccionario. Debe permitir:

- Agregar contactos (nombre y número de teléfono).
- Buscar contactos.
- Eliminar contactos.
- Mostrar todos los contactos ordenados.
- Salir.

Temas que usa: diccionarios, listas, funciones, bucles y condicionales.

Ejercicio

Proyecto B: Conversor de unidades [Intermedio]

Crea un conversor que permita elegir entre varias conversiones (por ejemplo: kilómetros a millas, grados Celsius a Fahrenheit, kilos a libras, litros a galones). El menú se repite hasta que el usuario elija salir. Temas que usa: funciones, condicionales, bucles, entrada de datos.

Ejercicio

Proyecto C: Quiz de cultura general [Intermedio]

Crea un cuestionario de 10 preguntas con 4 opciones cada una. El programa:

- Muestra cada pregunta y las opciones.
- Lee la respuesta del usuario.
- Cuenta los aciertos.
- Al final muestra el puntaje y una calificación.

Temas que usa: listas, diccionarios, bucles, condicionales, entrada de datos.

Ejercicio

Proyecto D: Generador de contraseñas [Difícil]

Crea un programa que genere contraseñas seguras. Debe:

- Preguntar la longitud deseada (por ejemplo, 12 caracteres).
- Incluir letras mayúsculas, minúsculas, dígitos y símbolos.
- Mezclar los caracteres al azar con `random.shuffle`.
- Mostrar la contraseña generada.

Temas que usa: módulo `random`, listas, cadenas, bucles, funciones.

Ejercicio

Proyecto E: Registro de notas [Difícil]

Crea un programa que lleve el registro de las notas de varios estudiantes. Debe:

- Permitir registrar estudiantes con sus notas.
- Permitir ver las notas de un estudiante.
- Calcular el promedio del curso.
- Mostrar un reporte con el promedio de cada estudiante.

Temas que usa: diccionarios, listas, funciones, bucles, condicionales.

12.2. Criterios de evaluación

El proyecto se evalúa con los siguientes criterios:

Criterio	Descripción
Funcionalidad	El programa hace lo que se pide y no se cae con errores
Claridad	El código es legible y usa nombres descriptivos
Comentarios	El código tiene comentarios que explican las partes importantes
Uso de conceptos	Combina al menos tres temas del curso correctamente
Manejo de errores	El programa no falla con entradas inesperadas
Creatividad	Aporta ideas o funciones adicionales a lo pedido

13. Referencias

- Documentación oficial de Python: <https://docs.python.org/es/3/>
- Página de descargas de Python: <https://www.python.org/downloads/>
- Python Tutor (visualizador de código): <https://pythontutor.com/>
- Replit (editor en línea): <https://replit.com/>

- Google Colab: <https://colab.research.google.com/>